AMSS 2026 — Lecture 8: Patterns I — Selection

Traian-Florin Șerbănuță

2026

Today's Agenda

1. Frame: from modeling to designing
2. The vocabulary: what a pattern is, three categories
3. The selection question: when a pattern earns its place
4. Live demo: drive AI to suggest patterns
5. Reading critically: where AI overuses and decorates
6. Bridge to Week 9; note this week's Lab 4

Recap: Architect and Critic

- **Architect / director:** drive AI through the software development lifecycle (SDLC).
- **Critic / reviewer:** read AI's output and name what's wrong or missing.

In Weeks 4-7 you critiqued AI's *models*. Today: the same loop, on a *design decision* — which patterns to use.

From Modeling to Designing

- **Weeks 4-7 pinned what the system is and does** — classes, interactions, lifecycles.
- **Today: how to structure the solution well** — design patterns, the named solutions to recurring problems.

Today's question: **does this pattern solve a real problem here — or is it decoration?**

The Literacy Floor: Critique, Rationale, Traceability

From Week 1: in the oral defense, *unaided*, you must demonstrate:

- **Critique** — read & critique AI-generated artifacts on the spot.
- **Rationale** — articulate why you directed AI a certain way.
- **Traceability** — defend traceability across your project.

Today is **Rationale** at its sharpest: a pattern is only as good as the *reason* for it. "Why this pattern, or why none?" is the question the defense asks.

What Is a Design Pattern?

A **named, reusable solution to a recurring problem in a context** (Gang of Four). Three parts:

- **Problem** — the recurring situation it addresses.
- **Solution** — the structure that resolves it.
- **Consequences** — what it costs (always some indirection).

A pattern is a tool for a problem — **not a goal in itself.**

Three Categories

- **Creational** — how objects are made: Factory, Builder, Singleton.
- **Structural** — how objects are composed: Adapter, Decorator, Composite, Proxy.
- **Behavioral** — how objects interact: Strategy, Observer, State, Command.

A **vocabulary** for naming solutions — not a checklist to apply.

The Vocabulary in Bike-Sharing

Where these *might* fit our domain:

- **Strategy** — interchangeable fare rules (peak / off-peak / member).
- **Observer** — riders notified when a station has bikes.
- **State** — the `Bike` lifecycle from Week 7 (Available, Reserved, InUse…).
- **Factory** — creating the right `Payment` (cash / card).

Recognise the shape. Whether each is *warranted* is the next question.

The Selection Question

Before applying any pattern, ask:

1. Is there a **recurring** problem — or a one-off?
2. Is there real **variation or change** to absorb?
3. Does the pattern's **intent** match this problem?
4. Is the **indirection worth it**?

If you can't answer these, you don't have a pattern — you have decoration.

Your Turn: Does It Earn Its Place?

AI proposes **Strategy + Factory + Singleton + Observer** for a per-minute fare calc with peak / off-peak / member rates.

With your neighbour (2 min): which survive the four questions? Be ready to defend **one keep** and **one cut**.

A Pattern Is a Cost

Every pattern adds **indirection** — more classes, more hops to follow.

- **Worth it** when it absorbs real, recurring variation.
- **Pure cost** when it doesn't — harder to read, nothing gained.

The question is never *"which pattern?"* It's *"is a pattern warranted at all?"*

Demo: Drive AI to Suggest Patterns

Live: ask AI which patterns to use for fare calculation. Watch how many it proposes — and whether each solves a problem that exists.

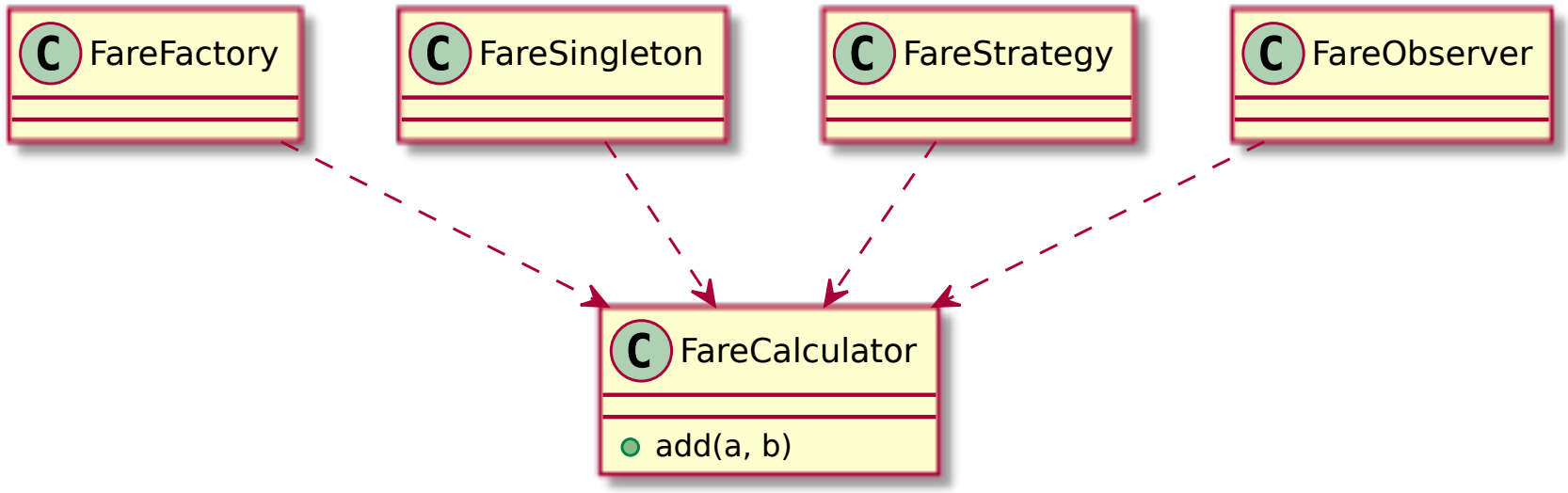
Prompt to AI: *"What design patterns should I use to implement fare calculation in the bike-sharing app? Rides are charged per minute, with peak / off-peak rates and a member discount."*

AI Reaches for Patterns — and Decorates

AI proposes patterns fluently, because patterns signal "good design." But a pattern with no problem behind it is decoration. The critic question:

> *"What problem does this pattern solve here — and does that problem actually exist?"*

Defect #1: Overuse



Four patterns to add two numbers. **Critique:** *"What recurring problem does each solve? Strip every pattern whose problem isn't here."*

Defect #2: Decoration — Labeled, Not Applied

A method with an `if peak / else off-peak` branch, **called "the Strategy pattern"** — but no `FareStrategy` interface, no interchangeable implementations.

Critique: *"Where's the Strategy interface? The interchangeable classes? A name isn't a pattern."*

Defect #3: Wrong Pattern

- **Singleton** for the fare config — creates hidden global state, untestable.
- **Observer** where a direct method call would do — indirection for nothing.

Critique: *"Does the pattern's intent match the problem — or just its vocabulary?"*

Defect #4: Premature Abstraction

A `PaymentFactory` and a `PaymentBuilder` — for a single concrete `CardPayment`. No second type exists yet.

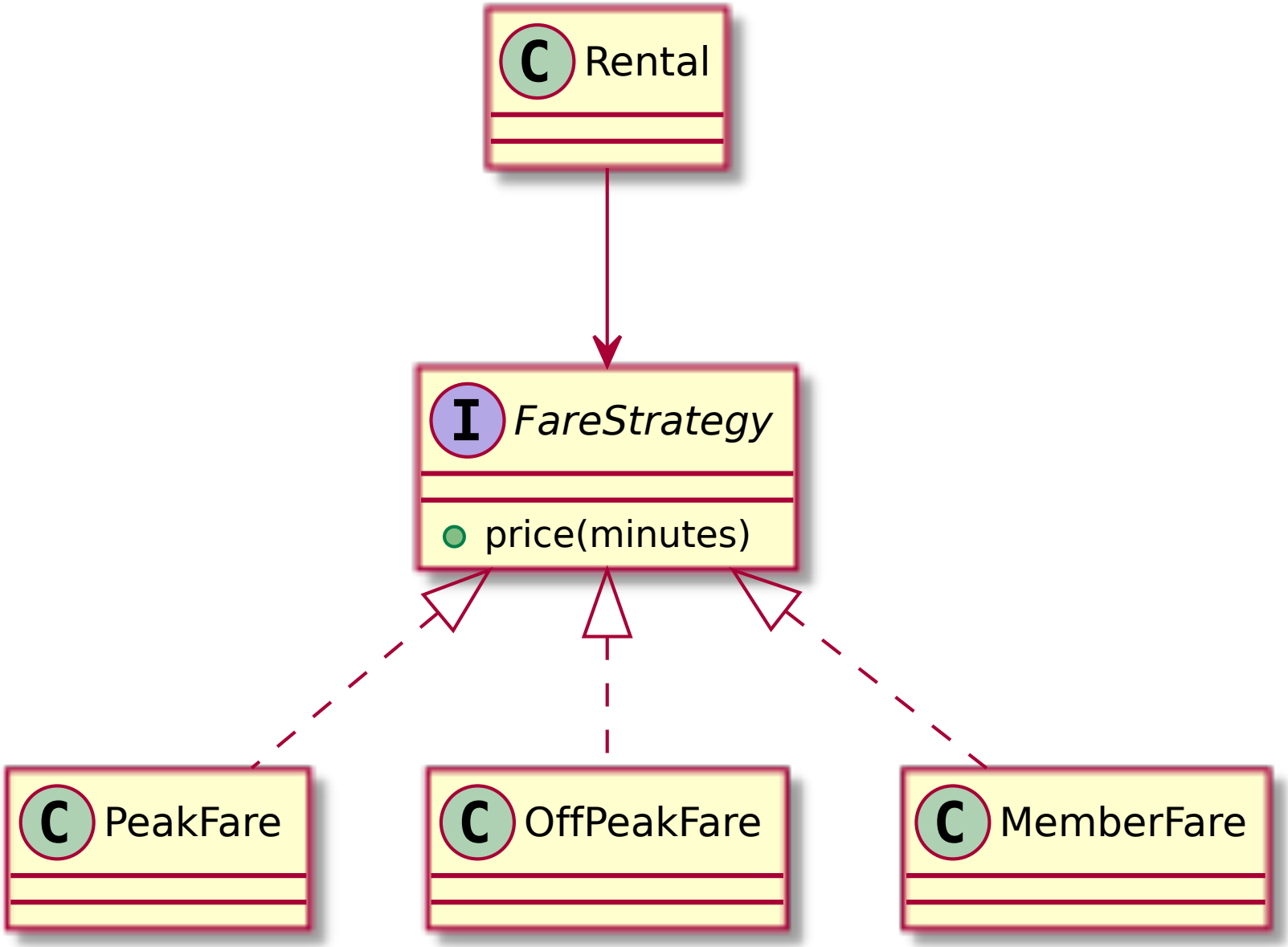
Critique: *"What variation does this absorb today? Build the abstraction when the second case arrives, not before."*

Defect #5: Mislabeling

A `PaymentFactory` that's actually a **Builder**; an `Adapter` called a **Facade**.

Critique: *"Name it correctly. If the name is wrong, the rationale is hollow — and so is the traceability."*

The Critic's Selection



One pattern, justified: fare rules really vary (peak / off-peak /

How to Read AI's Pattern Suggestions

A fixed order, every time:

1. What **problem** does each pattern claim to solve?
2. Does that problem **exist here** — recurring, with real variation?
3. Is the pattern **actually applied** (the structure), or just **labeled**?
4. Is the **indirection worth it**?

The Critique Loop on Design

read -> name the overuse / decoration -> re-prompt “name the recurring problem first, then the pattern” -> re-read.

Same architect-and-critic loop as Weeks 2-7 — now on a design decision.

The Human Decides Whether a Pattern Earns Its Place

AI will apply a pattern to anything. Whether the problem *warrants* one — whether the indirection buys something — is the architect-and-critic call.

It's what the oral defense checks, and AI can't make it for you.

This Week's Lab: Lab 4

Lab 4 runs this week — a **team defect hunt on flawed behavioural artifacts** (sequence, state, activity) from Weeks 6-7. Bring your behaviour read-orders.

Today's patterns content feeds **Week 9** and your project's design narrative, not this week's lab.

Next Week: Patterns II (Week 9)

Today was *whether* to use a pattern. Next week is *which*, in depth.

Week 9: Visitor, Mediator, Bridge, Adapter, Decorator, Proxy, Composite — and the deeper critique: **was this pattern applied, or just labeled?**

Critique, Rationale, Traceability — The Through-Line

Today you drilled:

- **Rationale** — justified *why* a pattern (or none): named the problem before the solution.
- **Critique** — read AI's pattern suggestions and named overuse, decoration, mislabeling.

Traceability: a correctly-named, justified pattern keeps the trace from problem to solution honest.

That's It For Today

- Next lecture (Week 9): patterns II — the specific patterns + applied-vs-labeled critique.
- This week's lab (Lab 4): team defect hunt on flawed behavioural artifacts.

Questions?